PREDICTIVE MAINTENANCE

Class Imbalance - Real Practice Exercise Guide

Dataset: UCI AI4I 2020 Predictive Maintenance

Mission

Build and justify a binary classification solution that detects machine failures in a strongly imbalanced industrial dataset. Your goal is not merely to obtain a high score. Your goal is to show that the model catches failures, understand the cost of mistakes, and select a decision threshold that makes sense for maintenance operations.

What this PDF gives you

The subject and business problem
Exactly what you must build and release
The required ML workflow
How to interpret every important metric
What conclusions are valid and what conclusions are not
A final delivery checklist and evaluation rubric

Practice first. Do not copy a previous solution line by line.

1. Business Case and Dataset

Business scenario. You work with a manufacturing maintenance team. Machines generate operating measurements such as temperatures, rotational speed, torque and tool wear. The team wants an early warning model that estimates whether a machine is likely to fail.

Business priority. A missed real failure can lead to downtime, lost production and emergency maintenance. Therefore, False Negatives deserve special attention. False Positives also matter because unnecessary inspections consume time and money.

The core decision problem

You are not trying to maximize accuracy at any cost. You are trying to find a useful balance between detecting real failures and avoiding excessive false alarms.

Dataset facts

Item	Value
Dataset	AI4I 2020 Predictive Maintenance (UCI)
Observations	10,000
Predictive features used here	6
Target	Machine failure: 0 = no failure, 1 = failure
Class distribution	9,661 non-failures vs 339 failures
Failure prevalence	3.39%
Missing values reported by UCI	No

Why this is a class-imbalance problem. Only about 3.39% of observations belong to class 1. A trivial model that predicts class 0 for everything can therefore look impressive on accuracy while being useless for failure detection.

Expected majority baseline intuition

If a model predicts class 0 for every observation, its accuracy can be close to 96.6%, yet recall for class 1 is 0%. This is exactly why accuracy alone is dangerous here.

Features you may use

- Type - product quality variant (categorical)
- Air temperature [K]
- Process temperature [K]
- Rotational speed [rpm]
- Torque [Nm]
- Tool wear [min]

2. Data Leakage Rules - Read This Before Coding

Critical rule

Do NOT use the failure-mode target columns TWF, HDF, PWF, OSF or RNF as input features when predicting Machine failure. Machine failure is determined from those failure modes, so using them would give the model information derived directly from the target. That is target leakage.

Also exclude identifiers. Do not use UID as a predictive feature. Do not use Product ID as a raw serial-like identifier. Use Type as the meaningful categorical product-quality feature.

Required X and y

```
numerical_features = [
    "Air temperature [K]",
    "Process temperature [K]",
    "Rotational speed [rpm]",
    "Torque [Nm]",
    "Tool wear [min]",
]

categorical_features = [
    "Type",
]

target = "Machine failure"
```

Why leakage matters. A leaked model can show spectacular metrics because it has effectively been given part of the answer. That is not predictive modeling; it is an expensive way to rediscover the label.

Learning objectives

- Recognize class imbalance from target counts and proportions.
- Build a majority-class baseline before trusting a real model.
- Compare normal Logistic Regression with class_weight="balanced".
- Understand exactly where class weighting changes training.
- Understand exactly where threshold changes affect final predictions.
- Interpret precision, recall, F1 and the confusion matrix in business terms.
- Use ROC-AUC and Average Precision as threshold-independent ranking summaries.
- Select and justify a threshold based on an explicit operational requirement.

3. What You Must Release

Recommended repository folder:

```
02-ML/
└── 10-predictive-maintenance-practice/
    ├── ai4i2020.csv
    ├── predictive_maintenance.py
    ├── README.md
    └── results/
        ├── metrics_comparison.csv
        ├── roc_curve.png
        └── precision_recall_curve.png
```

Minimum required deliverables

- predictive_maintenance.py - complete executable ML workflow.
- README.md - business problem, methodology, metrics, results and final recommendation.
- Dataset file or clear loading instructions/source reference.
- metrics_comparison.csv - one compact table comparing baseline, normal, balanced and selected-threshold results.
- roc_curve.png - ROC curve for the selected probability model.
- precision_recall_curve.png - Precision-Recall curve for the selected probability model.

Your code must run from the project folder with:

```
uv run python predictive_maintenance.py
```

README final recommendation. Your README must finish with a short recommendation to the maintenance team. It must state which model you choose, which threshold you choose, what recall/precision trade-off you accept, and why that trade-off makes operational sense.

> **Do not release a score dump**
>
> A real ML deliverable is not just twenty printed decimals. Every important result must be interpreted in plain business language.

4. Required ML Workflow

1. Inspect the target: Print class counts and normalized proportions. State which class is majority and which is minority.

2. Create X and y: Use only the six allowed predictive features. Exclude IDs and failure-mode targets.

3. Split safely: Use train_test_split with stratify=y so train and test keep approximately the same class proportions.

4. Build preprocessing: Scale numerical features and encode Type. Keep preprocessing inside a Pipeline.

5. Create DummyClassifier baseline: Use strategy="most_frequent". Evaluate at least accuracy, recall and confusion matrix.

6. Train normal Logistic Regression: Start with class_weight=None. Evaluate accuracy, precision, recall, F1 and confusion matrix.

7. Train balanced Logistic Regression: Use class_weight="balanced". Compare the new metrics with the normal model.

8. Get class-1 probabilities: Use predict_proba(X_test)[:, 1]. These are P(Machine failure = 1).

9. Test thresholds: Evaluate at least 0.5, 0.4, 0.3 and 0.2. Record precision, recall, F1, FP and FN.

10. Evaluate ranking quality: Calculate ROC-AUC and Average Precision from probabilities, not hard class predictions.

11. Choose an operating threshold: Find a threshold where recall >= 0.80 while keeping precision as high as possible among acceptable thresholds.

12. Write the business conclusion: Explain which errors remain, what you gain, what you sacrifice and why your final choice is reasonable.

5. Exact Before vs After: Class Weighting

Before balancing

```
LogisticRegression(
    class_weight=None
)
```

Conceptually, every class receives weight 1. If class 0 has far more rows, its observations dominate the total training loss simply because there are many more of them.

After balancing

```
LogisticRegression(
    class_weight="balanced"
)
```

Scikit-learn assigns class weights inversely proportional to class frequency. The formula is:**weight_c = N / (K * N_c)**

Where N is the number of training observations, K is the number of classes, and N_c is the number of observations in class c.

Where the change happens

class_weight="balanced" changes the loss used during fit(). It does not duplicate rows, does not modify X or y, and does not directly change the prediction threshold. Different weighting can lead to different learned coefficients, intercepts, probabilities and final predictions.

Interpretation template

```
The balanced model increased failure recall from ___ to ___.
This means it detected a larger proportion of real failures.
Precision changed from ___ to ___, meaning false alarms [increased/decreased].
The trade-off is [acceptable/not acceptable] because ___.
```

What not to conclude

- Balanced does not mean the dataset becomes 50/50.
- Balanced does not mean the model will automatically be better on every metric.
- A drop in accuracy can be acceptable if failure detection improves enough.

6. Exact Before vs After: Decision Threshold

Logistic Regression produces a probability for class 1. The threshold converts that probability into a final class.

```
probability = P(machine failure = 1)

if probability >= threshold:
   prediction = 1
else:
   prediction = 0
```

Example

```
P(class 1) = 0.32

threshold = 0.50  -> prediction = 0
threshold = 0.30  -> prediction = 1
```

Where the change happens

Changing the threshold does not retrain the model. The coefficients and probabilities stay the same. Only the final probability-to-class decision changes.

Expected direction when lowering the threshold:

- More observations are predicted as class 1.
- True Positives often increase.
- False Negatives often decrease.
- Recall usually increases.
- False Positives may increase.
- Precision may decrease.

Interpretation template

```
At threshold 0.50, recall = ___ and precision = ___.
At threshold 0.30, recall = ___ and precision = ___.
Lowering the threshold detected ___ more real failures,
but created ___ additional false alarms.
For this maintenance context, this trade-off is [acceptable/not acceptable] because ___.
```

7. Metrics: What Each Number Actually Means

Metric	Formula / basis	Plain meaning	Interpretation here
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall fraction of correct predictions.	Dangerous alone here because class 0 is about 96.6% of the data.
Precision	TP / (TP + FP)	Among predicted failures, how many really failed?	High precision means fewer unnecessary maintenance alarms.
Recall	TP / (TP + FN)	Among real failures, how many did we catch?	Critical when missed failures are expensive.
F1	2 * Precision * Recall / (Precision + Recall)	Single score balancing precision and recall.	Useful when you need one threshold-dependent summary, but business costs still matter.
ROC-AUC	Area under ROC curve	How well probabilities rank failures above non-failures across thresholds.	0.5 is random-like ranking; 1.0 is perfect ranking. It does not choose your threshold.
Average Precision	Summary of Precision-Recall relationship	How well the model ranks positives while preserving useful precision/recall behavior.	Especially informative for rare positive classes. Compare it with the positive-class prevalence.

Important Average Precision intuition. For a rare positive class, a naive ranking baseline is roughly tied to the positive prevalence. Here failures are about 3.39%, so an Average Precision meaningfully above that baseline is much more informative than simply saying the score is above zero.

8. Confusion Matrix: Translate It Into Maintenance Language

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

TN - True Negative: Machine operates normally and model predicts no failure.

FP - False Positive: Model predicts failure, but machine does not fail. Cost: unnecessary inspection, intervention or downtime.

FN - False Negative: Machine really fails, but model predicts no failure. Cost: missed warning, potential production interruption and emergency repair.

TP - True Positive: Machine fails and model correctly raises the warning.

> **Primary business risk in this exercise**
>
> Treat False Negatives as the more dangerous error unless your final analysis provides a strong reason otherwise. Your threshold selection should therefore explicitly discuss how many failures remain missed.

How to compare models correctly

Model	Accuracy	Precision	Recall	F1	FP	FN
Dummy baseline	...	...	...	...	...	...
Normal LogisticRegression	...	...	...	...	...	...
Balanced LogisticRegression	...	...	...	...	...	...
Balanced + chosen threshold	...	...	...	...	...	...

Your interpretation should focus on how TP, FP and FN move between rows. The metric numbers are summaries of those changes, not magical independent objects.

9. ROC, Precision-Recall and Threshold Selection

ROC curve

For many thresholds, ROC compares True Positive Rate (Recall) against False Positive Rate. Lower thresholds generally move you toward more detected failures but also more false alarms.

ROC-AUC interpretation: Use it to judge ranking/separation quality across thresholds. Do not use ROC-AUC by itself to decide the final operating threshold.

Precision-Recall curve

For many thresholds, the Precision-Recall curve shows the direct trade-off between catching failures and keeping failure alerts trustworthy. Because failures are rare, this curve is especially useful in this project.

Average Precision interpretation: Higher is better. Compare it with the positive-class prevalence and with alternative models. It is not a replacement for the final threshold decision.

Threshold rule for this exercise

> **Operational requirement**
>
> Find a threshold where Recall >= 0.80. Among thresholds satisfying that requirement, prefer the one with the highest Precision. Then report the resulting FP, FN, Precision, Recall and F1.

Important modeling discipline. In a production-grade workflow, threshold selection should be performed on validation or cross-validation predictions, and the test set should remain untouched until final evaluation. For this focused exercise, you may follow the simplified threshold experiment requested, but state this limitation in the README.

10. Interpretation Guide for Likely Results

A. DummyClassifier has very high accuracy but recall = 0

Correct interpretation: the baseline exploits class frequency. It predicts the majority class and misses every failure. High accuracy does not mean useful failure detection.

B. Normal Logistic Regression keeps high accuracy but recall is low

Correct interpretation: the model performs well on abundant non-failure observations but still misses too many rare failures. Do not celebrate accuracy while FN remains large.

C. Balanced Logistic Regression increases recall and lowers precision

Correct interpretation: minority-class mistakes received more influence during training, so the model detects more real failures. The price is more false alarms. Decide whether that cost is operationally acceptable.

D. Lowering the threshold increases recall

Correct interpretation: class-1 predictions became easier to trigger. More real failures are captured, but some non-failures are also flagged. Compare the reduction in FN with the increase in FP.

E. ROC-AUC is strong but precision is modest

Correct interpretation: the model may rank failures reasonably well, but because positives are rare, the operating threshold can still produce many false alarms. Ranking quality and deployed decision quality are related but not identical.

F. Average Precision is much higher than 3.39%

Correct interpretation: the model provides meaningful positive-class ranking beyond the rare-class prevalence baseline. Still report threshold-specific precision and recall for the final decision.

11. README Structure You Should Deliver

- 1. Project title and one-paragraph business context
- 2. Dataset source and feature description
- 3. Target definition and class distribution
- 4. Data leakage exclusions
- 5. Preprocessing strategy
- 6. DummyClassifier baseline and interpretation
- 7. Normal Logistic Regression results and interpretation
- 8. Balanced Logistic Regression results and interpretation
- 9. Threshold comparison table
- 10. ROC-AUC and Average Precision
- 11. Chosen threshold and business justification
- 12. Limitations
- 13. Final recommendation

Final conclusion template

The dataset is strongly imbalanced, with approximately ___% failures.
The majority-class baseline achieved accuracy of ___ but recall of ___,
showing that accuracy alone is misleading.

Normal Logistic Regression achieved recall of ___ and precision of ___.
Using class_weight="balanced" changed recall to ___ and precision to ___.

After evaluating thresholds, I selected threshold = ___.
At this threshold:
- Recall = ___
- Precision = ___
- F1 = ___
- False Positives = ___
- False Negatives = ___

I recommend this configuration because ___.
The main remaining risk is ___.

12. Common Mistakes That Will Invalidate the Exercise

Using TWF/HDF/PWF/OSF/RNF as features: Target leakage. These failure modes are directly related to how Machine failure is defined.

Using only accuracy: A 96%+ accuracy model can still miss almost every failure.

Using predict() for ROC-AUC/AP: ROC-AUC and Average Precision should use class-1 scores/probabilities, not only hard 0/1 predictions.

Calling fit_transform on test data independently: Preprocessing must be learned from training data. Keep preprocessing in the Pipeline.

Forgetting stratify=y: With a rare class, random splits can distort class proportions and make evaluation noisier.

Saying balanced changes the threshold: It does not. It changes the training loss; threshold is a separate decision step.

Saying lower threshold is always better: Lower thresholds improve recall only by accepting more positive predictions, often including more false positives.

Choosing threshold after looking repeatedly at final test results: This can overfit the test set. In serious work, tune threshold on validation/CV data and use the test set once at the end.

13. Evaluation Rubric and Final Checklist

Area	Weight
Data understanding and leakage prevention	15%
Correct preprocessing and stratified split	10%
Dummy baseline	10%
Normal vs balanced Logistic Regression	15%
Metric interpretation and confusion matrices	15%
Threshold analysis	15%
ROC-AUC and Average Precision	10%
README business recommendation and limitations	10%

Final checklist

- The script runs without manual path edits.
- Class counts and proportions are printed.
- Only allowed predictive features are used.
- Train/test split is stratified.
- Dummy baseline is evaluated.
- Normal Logistic Regression is evaluated.
- Balanced Logistic Regression is evaluated.
- Confusion matrices are interpreted.
- Thresholds 0.5, 0.4, 0.3 and 0.2 are compared.
- ROC-AUC is reported from probabilities.
- Average Precision is reported from probabilities.
- A threshold satisfying Recall >= 0.80 is selected if possible.
- The final recommendation discusses both FN and FP.
- README includes limitations and leakage notes.
- Results files and curves are saved in results/.

14. Source and Reference

Primary dataset source: UCI Machine Learning Repository - AI4I 2020 Predictive Maintenance Dataset.

Dataset citation: AI4I 2020 Predictive Maintenance Dataset [Dataset]. (2020). UCI Machine Learning Repository. DOI: 10.24432/C5HS5C.

UCI page: https://archive.ics.uci.edu/dataset/601/ai4i

Facts used from UCI: 10,000 observations; six predictive features; no missing values reported; feature definitions and target roles; the dataset is synthetic and designed to reflect industrial predictive-maintenance data.

Class-distribution cross-check: 9,661 non-failure observations and 339 failure observations (3.39% failures), consistent with published analyses of the AI4I dataset.

Your goal for this exercise

Finish with a model decision you can defend. If your conclusion only says "accuracy is high", the exercise is not finished.